

DBMS – ORDERS AND ORDER_ITEMS MANAGEMENT

ShopSphere Online E-Commerce Database Management System

1. Database Design

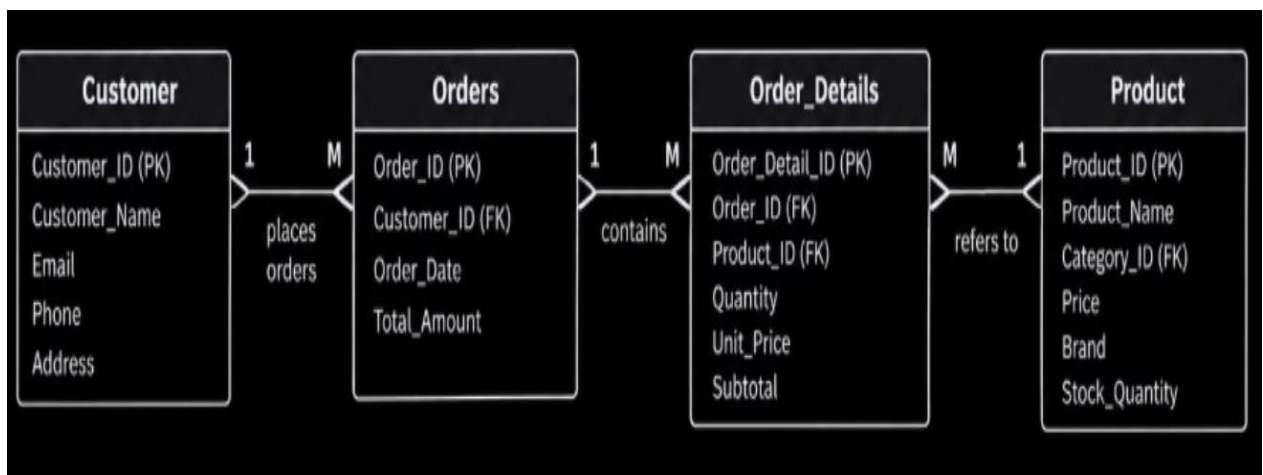
1.1 Orders Table

Attribute	Data Type	Constraint
Order_ID	INT	Primary Key
Customer_ID	INT	Foreign Key
Order_Date	DATE	NOT NULL
Total_Amount	NUMBER(10,2)	NOT NULL

1.2 Order_Details Table

Attribute	Data Type	Constraint
Order_Detail_ID	INT	Primary Key
Order_ID	INT	Foreign Key
Product_ID	INT	Foreign Key
Quantity	INT	NOT NULL
Unit_Price	NUMBER(10,2)	NOT NULL
Subtotal	NUMBER(10,2)	NOT NULL

2. Table Relationships



3. Table Creation

3.1 Create Orders Table

```
SQL> CREATE TABLE Orders (  
 2     Order_ID INT PRIMARY KEY,  
 3     Customer_ID INT NOT NULL,  
 4     Order_Date DATE NOT NULL,  
 5     Total_Amount NUMBER(10,2) NOT NULL,  
 6     CONSTRAINT fk_orders_customer  
 7         FOREIGN KEY (Customer_ID)  
 8         REFERENCES Customer(Customer_ID)  
 9 );
```

Table created.

3.2 Create Order_Details Table

```
SQL> CREATE TABLE Order_Details (  
 2     Order_Detail_ID INT PRIMARY KEY,  
 3     Order_ID INT NOT NULL,  
 4     Product_ID INT NOT NULL,  
 5     Quantity INT NOT NULL,  
 6     Unit_Price NUMBER(10,2) NOT NULL,  
 7     Subtotal NUMBER(10,2) NOT NULL,  
 8     CONSTRAINT fk_orderdetails_order  
 9         FOREIGN KEY (Order_ID)  
10         REFERENCES Orders(Order_ID),  
11     CONSTRAINT fk_orderdetails_product  
12         FOREIGN KEY (Product_ID)  
13         REFERENCES Product(Product_ID)  
14 );
```

Table created.

3.3 Order Data Insertion

Order Records

```
SQL> INSERT INTO Orders
  2 (Order_ID, Customer_ID, Order_Date, Total_Amount)
  3 VALUES
  4 (1001, 101, DATE '2026-09-15', 1499.00);

1 row created.

SQL>
SQL> INSERT INTO Orders
  2 (Order_ID, Customer_ID, Order_Date, Total_Amount)
  3 VALUES
  4 (1002, 102, DATE '2026-09-16', 2499.00);

1 row created.

SQL>
SQL> INSERT INTO Orders
  2 (Order_ID, Customer_ID, Order_Date, Total_Amount)
  3 VALUES
  4 (1003, 103, DATE '2026-09-17', 899.00);

1 row created.

SQL>
SQL> INSERT INTO Orders
  2 (Order_ID, Customer_ID, Order_Date, Total_Amount)
  3 VALUES
  4 (1004, 104, DATE '2026-09-18', 3298.00);

1 row created.

SQL>
SQL> COMMIT;

Commit complete.
```

4. Order_Details Data Insertion

```

SQL> INSERT INTO Order_Details
  2 (Order_Detail_ID, Order_ID, Product_ID, Quantity, Unit_Price, Subtotal)
  3 VALUES
  4 (1, 1001, 101, 1, 1499.00, 1499.00);

1 row created.

SQL> INSERT INTO Order_Details
  2 (Order_Detail_ID, Order_ID, Product_ID, Quantity, Unit_Price, Subtotal)
  3 VALUES
  4 (2, 1002, 102, 1, 2499.00, 2499.00);

1 row created.

SQL> INSERT INTO Order_Details
  2 (Order_Detail_ID, Order_ID, Product_ID, Quantity, Unit_Price, Subtotal)
  3 VALUES
  4 (3, 1003, 103, 1, 899.00, 899.00);

1 row created.

SQL> INSERT INTO Order_Details
  2 (Order_Detail_ID, Order_ID, Product_ID, Quantity, Unit_Price, Subtotal)
  3 VALUES
  4 (4, 1004, 104, 2, 1649.00, 3298.00);

1 row created.

SQL> COMMIT;

Commit complete.

```

5. Order_Details Records

```

SQL> SELECT * FROM Order_Details;

```

ORDER_DETAIL_ID	ORDER_ID	PRODUCT_ID	QUANTITY	UNIT_PRICE	SUBTOTAL
1	1001	101	1	1499	1499
2	1002	102	1	2499	2499
3	1003	103	1	899	899
4	1004	104	2	1649	3298

6. Order Modification Operations

6.1 Modify Order Amount

```
SQL> UPDATE Orders
  2  SET Total_Amount = 3598.00
  3  WHERE Order_ID = 1004;

1 row updated.
```

6.2 Modify Order Date

```
SQL> UPDATE Orders
  2  SET Order_Date = DATE '2026-09-20'
  3  WHERE Order_ID = 1003;

1 row updated.
```

7. Order_Details Modification

```
SQL> UPDATE Order_Details
  2  SET Quantity = 3,
  3      Subtotal = Quantity * Unit_Price
  4  WHERE Order_Detail_ID = 4;

1 row updated.
```

8. Customer Order History Report

```

SQL> SELECT
  2     c.Customer_ID,
  3     c.Customer_Name,
  4     o.Order_ID,
  5     o.Order_Date,
  6     od.Product_ID,
  7     od.Quantity,
  8     od.Unit_Price,
  9     od.Subtotal
10 FROM Customer c
11 JOIN Orders o
12     ON c.Customer_ID = o.Customer_ID
13 JOIN Order_Details od
14     ON o.Order_ID = od.Order_ID
15 ORDER BY c.Customer_ID, o.Order_Date;

```

CUSTOMER_ID	CUSTOMER_NAME				ORDER_ID
101	Arun Kumar				1001
15-SEP-26	101	1	1499	1499	
102	Priya Sharma				1002
16-SEP-26	102	1	2499	2499	
103	Rahul Raj				1003
20-SEP-26	103	1	899	899	

CUSTOMER_ID	CUSTOMER_NAME				ORDER_ID
104	Divya Sri				1004
18-SEP-26	104	3	1649	3298	

9. Customer-wise Order Summary

```

SQL> SELECT
  2     c.Customer_ID,
  3     c.Customer_Name,
  4     COUNT(o.Order_ID) AS Total_Orders,
  5     SUM(o.Total_Amount) AS Total_Spending
  6 FROM Customer c
  7 JOIN Orders o
  8     ON c.Customer_ID = o.Customer_ID
  9 GROUP BY c.Customer_ID, c.Customer_Name
 10 ORDER BY c.Customer_ID;

```

CUSTOMER_ID	CUSTOMER_NAME	TOTAL_ORDERS
101	Arun Kumar	1
	1499	
102	Priya Sharma	1
	2499	
103	Rahul Raj	1
	899	

CUSTOMER_ID	CUSTOMER_NAME	TOTAL_ORDERS
104	Divya Sri	1
	3598	

Conclusion

The Orders and Order_Details tables extend the existing ShopSphere Online E-Commerce Database Management System by managing customer orders, storing product quantities and prices, modifying order information, and generating customer order history and customer-wise order summary reports.